Cryptography in Action: An OpenSSL Lab

Information Security – Practical Report (TP)

Zineb Elansari
UM6P – College of Computing
October 20, 2026

---

1 Mission 1 — The RSA Password Challenge

Given: $p = 5$, $q = 11$, $e = 3$, $C = 23$.

Public and private keys.

$$n = pq = 5 \times 11 = 55$$
$$\varphi(n) = (p-1)(q-1) = 4 \times 10 = 40$$
$$ed \equiv 1 \pmod{40} \Rightarrow 3d \equiv 1 \pmod{40} \Rightarrow d = 27 \quad (3 \times 27 = 81 = 2 \times 40 + 1)$$

Public key: $(n, e) = (55, 3)$. Private key: $(n, d) = (55, 27)$.

Decryption. $m = C^d \bmod n = 23^{27} \bmod 55$. Using the Chinese Remainder Theorem: $23^{27} \equiv 3^{27} \equiv 2 \pmod 5$ and $23^{27} \equiv 1 \pmod{11}$, hence $m = 12$.

Verification. $m^e \bmod n = 12^3 \bmod 55 = 1728 \bmod 55 = 23 = C$.

Observation. Re-encrypting the recovered message gives back the intercepted ciphertext, so the recovered password is **12**. With such tiny primes, n is trivially factored, which makes RSA completely insecure.

2 Mission 2 — Break the Image, Not AES!

2.1 Challenge 1 — Investigate the evidence

The first 64 bytes of `classified.enc` were inspected (Figure 1).

- **Signature (8 bytes):** `53 61 6c 74 65 64 5f 5f` = ASCII `Salted__`.
- **Salt (8 bytes):** `57 5a b8 cf 60 19 ac 39`, which matches the `salt=575AB8CF6019AC39` printed by OpenSSL during decryption (Figure 3).

Answer to Question 1: 16 bytes (8-byte signature + 8-byte salt) must be excluded before analysing the ciphertext. The file is 205 232 bytes; $205232 - 16 = 205216$ bytes of ciphertext, i.e. the $432 \times 475 = 205\,200$ bytes of pixels plus one 16-byte padding block.

```
(base) C:\Users\WIN 11 25H2>cd /d "C:\Users\WIN 11 25H2\Desktop\OpenSSL_Lab"

(base) C:\Users\WIN 11 25H2\Desktop\OpenSSL_Lab>python -c "from pathlib import Path; data=Path('classified.enc').read_by
tes()[:64]; print(' '.join(f'{x:02x}' for x in data))"
53 61 6c 74 65 64 5f 5f 57 5a b8 cf 60 19 ac 39 77 75 1c 07 8d df 54 b3 8a 26 b7 f6 f4 a9 ce f0 77 75 1c 07 8d df 54 b3
8a 26 b7 f6 f4 a9 ce f0 77 75 1c 07 8d df 54 b3 8a 26 b7 f6 f4 a9 ce f0

(base) C:\Users\WIN 11 25H2\Desktop\OpenSSL_Lab>python -c "import os; print('Encrypted file size:', os.path.getsize('cla
ssified.enc'), 'bytes')"
Encrypted file size: 205232 bytes

(base) C:\Users\WIN 11 25H2\Desktop\OpenSSL_Lab>python -m pip install pillow
Requirement already satisfied: pillow in C:\Users\WIN 11 25H2\anaconda3\Lib\site-packages (12.3.0)

(base) C:\Users\WIN 11 25H2\Desktop\OpenSSL_Lab> from PIL import Image
'from' n'est pas reconnu en tant que commande interne
ou externe, un programme exécutable ou un fichier de commandes.

(base) C:\Users\WIN 11 25H2\Desktop\OpenSSL_Lab>

(base) C:\Users\WIN 11 25H2\Desktop\OpenSSL_Lab>notepad investigate.py

(base) C:\Users\WIN 11 25H2\Desktop\OpenSSL_Lab>python investigate.py
Encrypted file size: 205232
Total image blocks: 12825
Unique blocks: 739
Repeated block types: 232
SUCCESS: pattern_ecb.png created!
```

Figure 1: Inspection of the header, file size, and execution of `investigate.py`.

2.2 Challenge 2 — The encrypted image mystery

The program `investigate.py` skips the 16-byte header, splits the ciphertext into 16-byte blocks, and assigns the same grayscale value to identical blocks. The result is saved as `pattern_ecb.png`.

- Total image blocks: 12 825
- Unique blocks: 739
- Repeated block types: 232

What can be recognised? Without any password, the outline and structure of the original image are visible in `pattern_ecb.png`. In ECB mode, $C_i = E_K(P_i)$, so identical plaintext blocks produce identical ciphertext blocks. Only 739 distinct blocks exist among 12 825, which exposes the image's structure. This is a pattern leak, not a decryption of AES.

```
(base) C:\Users\WIN 11 25H2\Desktop\OpenSSL_Lab>python investigate.py
Encrypted file size: 205232
Total image blocks: 12825
Unique blocks: 739
Repeated block types: 232
SUCCESS: pattern_ecb.png replaced!
```

Figure 2: Re-run of `investigate.py`; `pattern_ecb.png` regenerated.

2.3 Challenge 3 — Unlock the original

The file was decrypted with the Mission 1 password (`12`):

```
openssl enc -d -aes-256-ecb -pbkdf2 -p -in classified.enc -out recovered.raw
```

OpenSSL 3.5.7 displayed the salt and the derived key (Figure 3). The recovered file is 205 200 bytes, exactly 432×475, and `recover_image.py` rebuilt `recovered.png` with Pillow.

What did the pattern not reveal? The ECB pattern only shows which blocks are equal, not their content. The actual pixel values (true gray levels, fine details, and the real content of the image) were visible only after decryption with the password.

2.4 Challenge 4 — Repair with CBC

4.1 `recovered.raw` was encrypted with AES-256-CBC into `classified_cbc.enc` (Figure 3); OpenSSL displays a fresh salt, key, and IV. The output is 205 232 bytes (Figure 4).

```
(base) C:\Users\WIN 11 25H2\Desktop\OpenSSL_Lab>openssl version
OpenSSL 3.5.7 9 Jun 2026 (Library: OpenSSL 3.5.7 9 Jun 2026)

(base) C:\Users\WIN 11 25H2\Desktop\OpenSSL_Lab>openssl enc -d -aes-256-ecb -pbkdf2 -p -in classified.enc -out recovered
.raw
enter AES-256-ECB decryption password:

salt=575AB8CF6019AC39
key=103518D2C327E91924A746FA172211D62AC098CD2BA94E2270CC1D3612D6F877

(base) C:\Users\WIN 11 25H2\Desktop\OpenSSL_Lab>python -c "import os; print('Recovered file size:', os.path.getsize('rec
overed.raw'), 'bytes')"
Recovered file size: 205200 bytes

(base) C:\Users\WIN 11 25H2\Desktop\OpenSSL_Lab>notepad recover_image.py

(base) C:\Users\WIN 11 25H2\Desktop\OpenSSL_Lab>python recover_image.py
Recovered bytes: 205200
Expected bytes: 205200
SUCCESS: recovered.png created!

(base) C:\Users\WIN 11 25H2\Desktop\OpenSSL_Lab>openssl enc -aes-256-cbc -salt -pbkdf2 -p -in recovered.raw -out classif
ied_cbc.enc
enter AES-256-CBC encryption password:

Verifying - enter AES-256-CBC encryption password:

salt=B79F53FC174BFBFA
key=12E9E6BF44198B5AA8D95325A38E7B0D52112C65B0A7BCD56075404DBE070892
```

Figure 3: ECB decryption, image recovery, and CBC encryption.

```
enter AES-256-CBC encryption password:

Verifying - enter AES-256-CBC encryption password:

salt=B79F53FC174BFBFA
key=12E9E6BF44198B5AA8D95325A38E7B0D52112C65B0A7BCD56075404DBE070892
iv =14594E93B9CA9EFBEA87F9AFD88FC95B

(base) C:\Users\WIN 11 25H2\Desktop\OpenSSL_Lab>python -c "import os; print('CBC file size:', os.path.getsize('classifie
d_cbc.enc'), 'bytes')"
CBC file size: 205232 bytes

(base) C:\Users\WIN 11 25H2\Desktop\OpenSSL_Lab>notepad compare_modes.py

(base) C:\Users\WIN 11 25H2\Desktop\OpenSSL_Lab>python compare_modes.py

Analyzing: classified.enc
File size: 205232
Total blocks: 12825
Unique blocks: 739
Repeated block types: 232
Extra repeated occurrences: 12086
Saved: pattern_ecb.png

Analyzing: classified_cbc.enc
File size: 205232
Total blocks: 12825
Unique blocks: 12825
Repeated block types: 0
Extra repeated occurrences: 0
```

Figure 4: CBC file size and first run of `compare_modes.py`.

4.2 `compare_modes.py` analyses both ciphertexts and saves `pattern_ecb.png` and `pattern_cbc.png` (Figure 5).

Metric	ECB	CBC
Total blocks	12 825	12 825
Unique blocks	739	12 825
Repeated block types	232	0
Extra repetitions	12 086	0

Table 1: ECB vs. CBC block comparison.

Explanation. In CBC, each plaintext block is XORed with the previous ciphertext block before encryption: $C_i = E_K(P_i \oplus C_{i-1})$, with a random IV for C_0. Identical plaintext blocks therefore give different ciphertext blocks, since the chaining input differs each time. CBC has no repeated blocks (all 12 825 are unique), so the original picture can no longer be recognised and the visual leakage disappears.

```
(base) C:\Users\WIN 11 25H2\Desktop\OpenSSL_Lab>notepad compare_modes.py

(base) C:\Users\WIN 11 25H2\Desktop\OpenSSL_Lab>python compare_modes.py

Analyzing: classified.enc
File size: 205232
Total blocks: 12825
Unique blocks: 739
Repeated block types: 232
Extra repeated occurrences: 12086
Saved: pattern_ecb.png

Analyzing: classified_cbc.enc
File size: 205232
Total blocks: 12825
Unique blocks: 12825
Repeated block types: 0
Extra repeated occurrences: 0
Saved: pattern_cbc.png

========== ECB vs CBC ==========
Total blocks       : ECB = 12825 | CBC = 12825
Unique blocks      : ECB = 739 | CBC = 12825
Repeated types     : ECB = 232 | CBC = 0
Extra repetitions  : ECB = 12086 | CBC = 0
```

Figure 5: Final ECB vs. CBC comparison output.

3 Conclusion

Mission 1 recovered the password 12 using RSA mathematics. Mission 2 showed that AES-256-ECB leaks the structure of the data even though AES itself is not broken, while AES-256-CBC removes this leakage thanks to chaining and a random IV.